

```

"""
JADS Agent – Telegram Bot
Dipendenze: pip install python-telegram-bot anthropic
"""

import os
import logging
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    CallbackQueryHandler, ContextTypes, filters
)
import anthropic

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# — CONFIG —
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY")

client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

# — SYSTEM PROMPTS —
SYSTEM_PROMPTS = {
    "brief": """Sei JADS Agent, l'assistente AI di JADS, content creation agency

Guida Simone in 3 fasi:

FASE 1 – BRIEF: Raccogli info chiave. Chiedi: cliente, settore, obiettivo, tone o
FASE 2 – STRATEGIA: Proponi 2-3 format video con motivazione, tono visivo, hook n
FASE 3 – COPY PRONTO: Genera 3 caption Instagram (con emoji e hashtag), 2 titoli
Tono: diretto, professionale, creativo. Lingua: italiano. Usa *grassetto* con ast

"clienti": """Sei JADS Agent, specializzato nella ricerca clienti per JADS, c

Quando Simone descrive un tipo di cliente, fornisci:
1. Lista di 5-8 lead specifici (brand reali milanesi o tipologie realistiche)
2. Per ogni lead: nome, settore, perché ha bisogno di content, presenza social st
3. Dove cercarli: hashtag Instagram, zone Milano, query Google Maps precise
4. Strategia di approccio consigliata

```

Sii specifico e pratico. Lingua: italiano. Usa **grassetto** con asterischi per Tel

"outreach": ""Sei JADS Agent, esperto di outreach per agenzie creative itali

Quando hai info su un lead, genera:

1. **DM Instagram** (max 5 righe, riferimento specifico al loro lavoro, no template)
2. **Email** (oggetto + corpo, professionale ma giovane)
3. **Follow-up** da mandare dopo 7 giorni di silenzio

I messaggi devono posizionare JADS come partner creativo, non solo editor.

Se mancano info, chiedi prima: chi è, cosa fanno, cosa hai notato online.

Lingua: italiano. Usa **grassetto** con asterischi per Telegram.""",

"portfolio": ""Sei JADS Agent. Aiuta Simone a gestire il portfolio di JADS.

Puoi:

- Registrare nuovi progetti (chiedi: titolo, cliente, tipo di contenuto, link)
- Elencare i progetti salvati
- Suggestire come presentare il portfolio a potenziali clienti
- Creare una bio/presentazione di JADS basata sui progetti

Lingua: italiano.""

}

WELCOME_MSG = ""👋 Ciao Simone! Sono il tuo **JADS Agent**.

Scegli cosa vuoi fare: ""

```
MODE_LABELS = {  
    "brief": "🎯 Brief & Copy",  
    "clienti": "🔍 Cerca Clienti",  
    "outreach": "💬 Outreach",  
    "portfolio": "📁 Portfolio"  
}
```

```
MODE_HINTS = {  
    "brief": "📝 *Brief & Copy attivo*\n\nDimmi tutto sul prossimo progetto: clie  
    "clienti": "🔍 *Ricerca clienti attiva*\n\nDimmi settore, zona e tipo di busi  
    "outreach": "💬 *Outreach attivo*\n\nDimmi chi vuoi contattare: nome, cosa fa  
    "portfolio": "📁 *Portfolio attivo*\n\nPosso registrare nuovi progetti, elenc  
}
```

— STATE (in-memory, per utente) —————

user_states = {} # { user_id: { mode, history, portfolio } }

def get_state(user_id):

```

    if user_id not in user_states:
        user_states[user_id] = {
            "mode": "brief",
            "history": {m: [] for m in SYSTEM_PROMPTS},
            "portfolio": []
        }
    return user_states[user_id]

# — KEYBOARD —————
def mode_keyboard():
    return InlineKeyboardMarkup([
        [
            InlineKeyboardButton("📋 Brief & Copy", callback_data="mode_brief"),
            InlineKeyboardButton("🔍 Clienti", callback_data="mode_clienti"),
        ],
        [
            InlineKeyboardButton("💬 Outreach", callback_data="mode_outreach"),
            InlineKeyboardButton("📁 Portfolio", callback_data="mode_portfolio"),
        ]
    ])

def back_keyboard():
    return InlineKeyboardMarkup([[
        InlineKeyboardButton("🔄 Cambia modalità", callback_data="show_menu")
    ]])

# — HANDLERS —————
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        WELCOME_MSG,
        parse_mode="Markdown",
        reply_markup=mode_keyboard()
    )

async def button_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    user_id = query.from_user.id
    state = get_state(user_id)

    if query.data == "show_menu":
        await query.message.reply_text(
            WELCOME_MSG,
            parse_mode="Markdown",
            reply_markup=mode_keyboard()
        )
    return

```

```

if query.data.startswith("mode_"):
    mode = query.data.replace("mode_", "")
    state["mode"] = mode
    await query.message.reply_text(
        MODE_HINTS[mode],
        parse_mode="Markdown",
        reply_markup=back_keyboard()
    )

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    state = get_state(user_id)
    text = update.message.text
    mode = state["mode"]

    # Add to history
    history = state["history"][mode]
    history.append({"role": "user", "content": text})

    # Show typing
    await context.bot.send_chat_action(
        chat_id=update.effective_chat.id,
        action="typing"
    )

    try:
        # Build messages (keep last 20 to avoid token overflow)
        messages = history[-20:]

        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1000,
            system=SYSTEM_PROMPTS[mode],
            messages=messages
        )

        reply = response.content[0].text

        # Save assistant response
        history.append({"role": "assistant", "content": reply})

        # Send reply (split if too long for Telegram)
        if len(reply) > 4000:
            chunks = [reply[i:i+4000] for i in range(0, len(reply), 4000)]
            for chunk in chunks:
                await update.message.reply_text(chunk, parse_mode="Markdown")

```

```

        else:
            await update.message.reply_text(
                reply,
                parse_mode="Markdown",
                reply_markup=back_keyboard()
            )

    except Exception as e:
        logger.error(f"Error: {e}")
        await update.message.reply_text(
            "⚠ Errore nella risposta. Riprova tra un secondo.",
            reply_markup=back_keyboard()
        )

async def reset(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    state = get_state(user_id)
    mode = state["mode"]
    state["history"][mode] = []
    await update.message.reply_text(
        f"🔄 Chat *{MODE_LABELS[mode]}* resettata.",
        parse_mode="Markdown"
    )

async def menu(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        WELCOME_MSG,
        parse_mode="Markdown",
        reply_markup=mode_keyboard()
    )

# ——— MAIN ———
def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("menu", menu))
    app.add_handler(CommandHandler("reset", reset))
    app.add_handler(CallbackQueryHandler(button_callback))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

    logger.info("JADS Agent avviato ✅")
    app.run_polling()

if __name__ == "__main__":
    main()

```

